

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)		
Student Name:	Dhanush Kumar R	Reg. No.:	192411215
Section / Batch:		Date:	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I Dhanush Kumar B (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Dhanush Kumar B

RUBRICS FOR CALCULATION

Criteria	Wt	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)	4	3	3	3	4	41
Q2 (50 Marks)	3	4	3	3	4	43
Overall Total (100)						85

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Name: Dhanush Kumar.R

Reg No: 192411215

COI-AT 1

Course Code: CSAD618

Course Name: Design and Analysis of Algorithm

1) Application : Online Video Streaming Platform

Given :-

$$T(n) = 2T(n/2) + n$$

Step 1: Identify Parameters

* $a = 2$

* $b = 2$

* $f(n) = n$

Step 2: Calculate

* $n^{\log_b a} = n^{\log_2 2} = n^{1+0} = n$

Since,

$$f(n) = O(n^{\log_b a})$$

* This matches Master Theorem Case 2

Step 3: Time complexity

* $T(n) = O(n \log n)$

Step 4: Efficiency Analysis

* The algorithm divides user data into two equal parts

* Each level performs linear work

* There are $\log n$ recursion levels

* Hence total work becomes $n \log n$

Scalability

* As the number of users increases, the algorithm grows efficiently and scales much better than quadratic algorithms

Final Answer:

* Time Complexity : $\Theta(n \log n)$

2) Application : Social Media Feed Processing

Given

$$T(n) = T(n-1) + n$$

Step 1: Expand using substitution

$$T(n) = T(n-1) + n$$

$$= T(n-2) + (n-1) + n$$

$$= T(n-3) + (n-2) + (n-1) + n$$

Continue until

$$= T(1) + 2 + 3 + \dots + n$$

Step 2: Sum

$$1+2+\dots+n = \frac{n(n+1)}{2}$$

Ignoring constants,

$$T(n) = O(n^2)$$

Performance Analysis:

- * Each recursive call processes one fewer element
- * Total work equals the sum of first n numbers
- * Running time increases rapidly for large

inputs

Final Answer:

- * Time Complexity: $O(n^2)$

3) Application : E-commerce Recommendation System

Algorithm 1

$$O(n \log n)$$

Algorithm 2

$$O(n^2)$$

Comparison

Applicat³

Algorithm	Growth Rate	Efficiency
$O(n \log n)$	Slower Growth	Better
$O(n^2)$	Faster Growth	Poor

Example

* For $n = 1000$

* Algorithm 1:

$$1000 \times \log_2 1000 \approx 10000$$

* Algorithm 2:

$$1000^2 = 1000000$$

* The quadratic algorithm performs far more operations

Conclusion:

* The $O(n \log n)$ algorithm is more efficient, less execution time for large datasets, and provides better scalability

Final Answer:

* Choose $O(n \log n)$ because it is faster and more scalable

Application : Cloud Storage Processing System

Given:

$$* T(n) = 3T(n/2) + n$$

Step 1: Parameters

$$* a = 3$$

$$* b = 2$$

$$* f(n) = n$$

Step 2: Calculate

$$* n \log_2^3 = n^{1.585}$$

$$\text{Since, } n < n^{1.585}$$

$$\text{Therefore } f(n) = 0$$

* This satisfies Master Theorem Case 1

Step 3: Time Complexity

$$* T(n) = \Theta(n \log_2^3) \approx \Theta(n^{1.585})$$

Performance Analysis

* Data is divided into three recursive tasks

* Growth is faster than $n \log n$

* Suitable for moderately large cloud storage systems

Final Answer:

$$* \text{Time complexity: } \Theta(n^{1.585})$$

5)

Application : Financial Analytic System

Given:-

$$T(n) = 2T(n/2) + n \log n$$

Step 1: Parameters

- * $a = 2$
- * $b = 2$
- * $f(n) = n \log n$

Step 2: Calculate

$$* n \log_2^2 = n$$

$$\text{compare, } f(n) = n \log n = O(n \log n)$$

$$\text{This equals } O(n \log^2 n)$$

* Therefore it follows Master Theorem, Case 2
(Extended form)

Step 3: Time complexity

$$* T(n) = O(n \log^2 n)$$

Performance Analysis:

- * Each recursive level performs $n \log n$ work
- * There are $\log n$ recursion levels
- * Total complexity becomes $n \log^2 n$.
- * More efficient than $O(n^2)$ for large datasets

Final answer:

$$* \text{Time complexity: } O(n \log^2 n)$$